

CYBER TREE

Complete Technical Analysis Report

Prepared automatically through source-code analysis

Version 0.1.0 | 17 July 2026

Scope: repository snapshot analyzed locally; live production claims are identified as historical evidence, not revalidated in this report.

Table of Contents

1. Introduction	
2. System Analysis	
3. System Requirements	
4. Complete System Architecture	
5. Module Analysis	
6. Source Code Analysis	
7. Database Analysis	
8. API Analysis	
9. Cybersecurity Analysis	
10. AI and Automation Analysis	
11. Workflow Documentation	
12. UI / UX Analysis	
13. Deployment Analysis	
14. Performance Analysis	
15. Software Testing	
16. Code Quality Review	
17. Future Roadmap	
18. Conclusion	
19. Bibliography and References	
Appendix A. Complete Project Map	
Appendix B. Verification and Assumptions	

1. Introduction

CyberTree is a cybersecurity threat-intelligence platform that converts external security reporting and curated vulnerability knowledge into a searchable graph. Its design combines scheduled Python acquisition and enrichment jobs with a Next.js dashboard and server-side API surface.

The intended outcome is an always-on research workspace: raw feeds and public standards data become typed knowledge nodes, nodes are related and embedded, downstream jobs identify clusters, trends and hypotheses, and analysts explore or query the resulting knowledge base.

The repository positions the platform for researchers, security analysts, students, and operators who need incident, vulnerability, actor, malware, technique, tool, weakness, news, and research context in one place. It is not a defensive control plane and does not perform endpoint monitoring, active scanning, incident response, or user/account administration.

1.1 Objectives and scope

Primary objectives are automated collection, node extraction and classification, relationship construction, semantic and full-text retrieval, visualization, trend and prediction reporting, health monitoring, and maintainable local development fallback.

Scope includes a public read-oriented dashboard, Next.js API routes, Supabase/PostgreSQL with pgvector in production, a SQLite fallback for local use, ML artifacts, and GitHub Actions scheduling. The scope excludes an identity provider, RBAC, tenant separation, authenticated write APIs, a formal API specification, test suite, Docker image, and IaC.

1.2 Key limitations

The report found an architectural mismatch between the public-facing threat-intelligence goal and the security model: all dashboard/API reads are anonymous and the supplied Supabase setup explicitly disables RLS. This can be acceptable for a deliberately public dataset, but it is not appropriate for private intelligence, multi-tenancy, or analyst-only findings.

Some repository documents contain historical audit data and completion logs. Those are useful evidence but may be stale; this report distinguishes current source behavior from those claims.

2. System Analysis

Existing approaches commonly split cyber intelligence between vendor feeds, vulnerability databases, ATT&CK/CWE/CAPEC taxonomies, dashboards, and analyst notes. CyberTree addresses fragmentation by normalizing material into a graph and exposing both browse-first and retrieval-first experiences.

Its proposed solution is technically feasible on the selected managed components: Vercel hosts the web application; Supabase supplies relational, JSON, full-text and vector features; GitHub Actions provides

inexpensive schedule-driven execution. The model is operationally feasible for periodic intelligence enrichment but relies on third-party availability, rate limits, and credentials.

Economic feasibility is favorable for a student/prototype system because it leans on managed/free tiers. Its main scalability constraint is not browser rendering but job time, embedding/model cost, external-feed rate limits, Supabase query volume, and the SQLite fallback's full in-process scans.

3. System Requirements

Area	Verified requirement / dependency	Role
Web runtime	Node.js compatible with Next.js 16.2.9; React 19.2.4	Dashboard and route handlers
Python runtime	Python 3.11 in GitHub Actions	Scheduled collection and intelligence scripts
Database	Supabase PostgreSQL + pgvector + FTS; SQLite local fallback	Primary persistence and local development
Web packages	@supabase/supabase-js, Recharts, Tailwind CSS 4	DB access, charting, UI styling
Python packages	supabase, httpx, feedparser, python-dotenv, numpy, scikit-learn, pytest	Network I/O, storage, ML and tooling
Additional workflow installs	sentence-transformers, torch	Offline embedding / clustering workflow
Hosting	Vercel configuration uses `npm run build` and `.next`	Next.js deployment

3.1 Environment variables

Variable	Consumers	Purpose / handling
SUPABASE_URL, SUPABASE_KEY	Python jobs	Server-side Supabase connection
SUPABASE_ANON_KEY / NEXT_PUBLIC_SUPABASE_*	Next.js db module	Browser-safe anonymous API connection; used server-side here too
GEMINI_API_KEY	/api/research, hypothesis engine, research_assistant.py	LLM generation; missing key yields explicit application error
HUGGINGFACE_API_KEY	Next.js and Python retrieval helpers	Optional API token; no token is permitted
OPENAI_API_KEY	process.py	Optional embedding call path
NVD_API_KEY	ingest_nvd.py	Higher-rate NVD API requests
GITHUB_TOKEN, GITHUB_REPOSITORY, GITHUB_BRANCH	backup/model updater	Release upload and model artifact commits
ALERT_WEBHOOK_URL	monitor.py	Health alert delivery
SQLITE_DB_PATH	Python DBClient	Optional local SQLite path override

4. Complete System Architecture

CyberTree has four cooperating planes: a presentation/API plane, a persistence/search plane, a scheduled intelligence plane, and external services. The database is the system boundary between web reads and batch writes. The user-facing API does not invoke batch scripts synchronously.

4.1 High-level architecture

```

flowchart LR
    U[Analyst Browser] --> W[Next.js UI + Route Handlers]
    W --> S[(Supabase PostgreSQL + pgvector + FTS)]
    W --> G[Gemini Research API]
    W --> H[Hugging Face embeddings]
    A[GitHub Actions schedules] --> P[Python intelligence pipeline]
    P --> S
    P --> X[Feeds, NVD, MITRE, CWE, CAPEC, CISA]
  
```

4.2 Request and knowledge lifecycle

Flow	Implementation	Output
Dashboard	React page fetches /api/stats and /api/system-health	Counts, jobs, health status
Browse/search	Routes call db.ts; optional query embedding then match_nodes RPC	Filtered node list or graph
Research	Question -> vector + FTS concurrently -> merge/rank -> Gemini with numbered context	Markdown answer and source IDs
Ingestion	collect.py stores raw_sources; process.py extracts content/classifies/inserts nodes	Typed graph candidates
Enrichment	relate, embed, cluster, trend, hypotheses, evidence, learning, maintenance	Relationships, vectors, trends, predictions, metrics

4.3 Architecture notes

`src/lib/db.ts` owns the data-access abstraction. When Supabase variables are available, it uses the remote database and RPCs; otherwise it opens `cyber\_tree\_local.db` through Node's `DatabaseSync`. This is a pragmatic development fallback, but query semantics differ: Supabase uses pgvector and generated FTS; SQLite performs JavaScript cosine similarity or SQL LIKE searches.

Archived synthetic nodes are filtered in many, but not all, read paths. This is an important cross-cutting policy encoded as `metadata.archived\_reason = synthetic-mock-data-phase13-removal`, rather than an explicit status column or view.

5. Module Analysis

Module group	Responsibilities	Principal dependencies / considerations
Next.js app and	Dashboard, route-specific UI, global search,	Client fetches; no user state or access control

Module group	Responsibilities	Principal dependencies / considerations
Header	health indicator, keyboard shortcut	
API route handlers	Thin JSON endpoint wrappers around db, classifier, embeddings, and Gemini	'force-dynamic'; broad catch-to-500 behavior
db.ts	Supabase/SQLite access, graph traversal, search, metrics, sources	Remote RLS/RPC/schema parity; local scans can be expensive
embeddings.ts	Hugging Face feature extraction with deterministic hash fallback	Fallback is dimension-compatible but semantically invalid
classifier.ts	Loads exported TF-IDF/logistic weights; keyword fallback	Synchronous model file read is cached; taxonomy consistency required
Python DBClient	Batching, upsert, raw-source state, job logs, SQLite schema	Queue flush semantics and backend portability
Pipeline scripts	Collection, processing, relation, vector, pattern, learning, maintenance	External API availability and scheduled execution order
GitHub workflows	Cron/manual orchestration, Python 3.11 setup, secret injection	No workflow dependency gating; independent schedules may race

6. Source Code Analysis

The TypeScript layer is intentionally thin. Pages generally own presentation and call read-only API endpoints. API handlers parse small query/body inputs, delegate to one library function, and return JSON. There are no controllers, repositories, ORM entities, middleware, queues, or authentication components in the conventional enterprise sense.

6.1 Frontend pages and components

File / route	Contribution
src/app/page.tsx	Client dashboard: counts, type distribution, recent nodes/jobs, health and quick navigation.
explore/page.tsx and explore/[id]/page.tsx	Graph viewer plus server-rendered node/relationship detail.
search/page.tsx and similar/[id]/page.tsx	Filtered/semantic discovery and nearest-neighbor view.
timeline/page.tsx; threats/page.tsx	Server-rendered incident and threat-actor catalog views.
patterns/page.tsx + charts.tsx	Trend/summary display and Recharts type/tag visualizations.
predictions, intelligence, sources, research pages	Prediction filters, learning metrics, source drill-down, and chat-style RAG UI.
components/Header.tsx; layout.tsx; globals.css	Navigation, global Ctrl/Cmd+K search, health indicator, metadata, global styling.

6.2 Core algorithms

Component	Actual algorithm
classifier.ts	Tokenizes word tokens, generates unigrams/bigrams, builds a TF-IDF vector, L2 normalizes, computes logistic-regression scores, applies stable softmax, returns argmax class/confidence. Keyword rules provide a fixed 0.70 fallback.
embeddings.ts	Calls Hugging Face all-MiniLM-L6-v2 inference API. On failure, SHA-256-seeded LCG yields a normalized 384-vector. It preserves shape but does not preserve semantic similarity.
research route	Parallel vector/FTS retrieval; vector similarity becomes base score; FTS overlap adds 0.2; FTS-only receives 0.7; top ten nodes form the LLM context.
relate.py	Uses embedding cosine similarity plus entity/keyword/co-occurrence logic to create relationship records; safe insertion is intended to prevent duplicate relations.
cluster_nodes.py	Fetches embeddings, derives clusters, writes cluster metadata; trend detector derives emergence signals from recent node populations.

6.3 Python script map

Scripts	Purpose
collect.py, process.py, db_client.py	RSS/advisory collection; article extraction/classification/embedding; backend-neutral storage queue.
relate.py, embed_nodes.py, cluster_nodes.py, trend_detector.py, pattern_report.py	Graph connections, vector backfill, clustering, trend records, and cluster summary report.
hypothesis_engine.py, evidence_linker.py, learning_engine.py, model_updater.py	LLM-guided predictions, evidence scoring, metrics/confidence adjustment, retraining and optional GitHub model update.
source_discoverer.py, dead_source_detector.py, knowledge_healer.py, maintain.py, monitor.py, backup.py	Source lifecycle, graph repair, archival maintenance, health monitoring, and backup release output.
ingest_nvd.py, ingest_mitre.py, ingest_cwe.py, ingest_capec.py, ingest_cisa.py	Structured external knowledge ingestion; some retain mock fallback generators.
export_training_data.py, train_classifier.py, relabel_and_train.py, classify.py, research_assistant.py, test_tech.py	Training-data extraction, classifier build/relabeling, CLI inference/research, and diagnostics.

7. Database Analysis

The primary schema defines raw_sources, nodes, relationships, sources, predictions, job_logs, and trends. The application also reads learning_metrics and some scripts call `update\_node\_metadata\_batch`; neither is defined in `schema.sql` or `supabase\_setup.sql`, so deployments need additional migrations not included in the repository. This is a documentation and reproducibility gap.

Table	Lifecycle / relationships
raw_sources	Append collection payload; checksum prevents duplicates; process state and errors are retained.
nodes	Canonical typed knowledge objects. Supports tags, confidence, source provenance, external IDs, JSON metadata and 384-d vectors.
relationships	Directed node-to-node edges with relation label, confidence, evidence, cascade delete foreign keys.
sources	Feed registry with reliability/activity/last-check status.
predictions	Hypotheses with related node IDs, evidence arrays, confidence, state and metadata.
job_logs	Start/finish/status/row counts/errors used by health UI and monitor.
trends	Cluster/trend/summary records with supporting node IDs and metadata.
learning_metrics (used, not migrated here)	Accuracy, confirmation and model-retraining history read by Intelligence UI.

Indexes cover raw-source checksum and work queue, node type/external ID/FTS, relationship endpoints/pairs, sources URL, job log ordering, trends, and predictions. The `match\_nodes` RPC exposes cosine distance over pgvector(384), filtering optional node type and applying a threshold.

8. API Analysis

Endpoint	Method	Input / workflow	Auth / response
/api/classify	POST	JSON `{text}` or raw body -> classifier	None; 400 blank, prediction JSON
/api/nodes	GET	type/search/tag/semantic/limit/offset/threshold -> db query	None; node list
/api/nodes/[id]	GET	Path ID -> node + relationships	None; 404 missing
/api/graph	GET	centerNodeId/depth -> traversal or default graph	None; graph nodes/edges
/api/similar/[id]	GET	threshold/limit -> pgvector matches	None; list; SQLite returns empty
/api/global-search	GET	q -> parallel-ish cross-entity ILIKE lookup	None; grouped top-five results
/api/stats, /api/system-health	GET	No arguments -> db aggregate/job-window evaluation	None; dashboard payload
/api/patterns, /api/intelligence	GET	No arguments -> trends/learning metrics	None; reporting payload
/api/predictions and /[id]	GET	status or path ID -> prediction data	None; 404 detail missing
/api/sources and /[id]	GET	Path ID lookup also samples matching nodes	None; 404 detail missing
/api/research	POST	`{question}` -> embedding + FTS + Gemini synthesis	None; in-memory

Endpoint	Method	Input / workflow	Auth / response
			10/min/IP limit; 429, 400, 500

All endpoints set `dynamic = force-dynamic`, preventing static caching. Validation is basic: numeric inputs are parsed but not bounded, and `/api/research` only checks a non-empty question. The standard error body exposes `err.message`, which risks leaking implementation details.

9. Cybersecurity Analysis

9.1 Implemented controls

Control	Evidence / assessment
Secrets via environment	Keys are read from environment and workflow secrets rather than committed source. Good baseline; no secret scanner/config validation is present.
Data provenance	Nodes retain source URL/name and confidence; research route returns source node IDs/titles for citations.
Input checks	Blank body/question handling; Next/Supabase parameterization avoids direct SQL string execution in normal paths.
Rate limit	Research endpoint uses a process-local 10 requests/minute map keyed by x-forwarded-for. This is not distributed or spoof-resistant.
Scheduled health	job_logs, monitor.py, system-health endpoint and optional alert webhook provide operational visibility.

9.2 Risks and recommendations

Priority	Finding	Recommendation
Critical	Supabase setup disables RLS for all core tables; no Next.js authentication/authorization exists.	Enable RLS; expose read-only views/RPCs; use server-only service role key for batch jobs; add analyst/admin identity and RBAC.
High	Public endpoints expose the corpus, sources, job health, prediction details, and LLM-backed endpoint without access control.	Decide public vs private data classification; require auth for operational and source-management data; add API authorization policy.
High	Embedding failure silently falls back to hash vectors, degrading relevance while appearing functional.	Fail closed for semantic features or mark degraded results; use a reliable hosted/local embedding service and telemetry.
High	Research prompt accepts arbitrary question text, and untrusted corpus text is placed into LLM context.	Add prompt-injection isolation, content delimiters, output schema/citation validation, moderation/risk policy, and audit logs.
Medium	No request-size caps, schema validation library, numeric range enforcement, or shared rate limiting.	Use Zod/Valibot, body limits, pagination/depth caps, Vercel/Redis rate limiting and abuse monitoring.

Priority	Finding	Recommendation
Medium	Errors return raw messages; logs may contain source/LLM details.	Use opaque public error IDs and structured, redacted server logs.
Medium	Workflow dependency order is schedule-based, not event/queue driven.	Use durable orchestration, idempotency keys, retries/backoff, and job dependency checks.

OWASP mapping: A01 Broken Access Control applies to disabled RLS and absent authorization; A02 Cryptographic Failures is mitigated only by platform HTTPS/secrets, with no app-level policy; A03 Injection covers LLM prompt injection and unbounded query shapes; A05 Security Misconfiguration includes public table policy and missing migration completeness; A09 Logging/Monitoring is partially addressed by job logs but lacks security audit logging.

10. AI and Automation Analysis

CyberTree implements AI-adjacent capabilities at several layers: supervised node typing, semantic embeddings, graph-based similarity, cluster/trend detection, Gemini question answering, and Gemini-assisted hypothesis generation. It is a retrieval-augmented generation system rather than an autonomous decision system; outputs are research aids whose confidence and citation quality must be verified by an analyst.

```

flowchart TD
    Q[Question] --> E[Query embedding]
    Q --> F[Full-text search]
    E --> V[pgvector match_nodes]
    F --> M[Merge + rank]
    V --> M
    M --> C[Top 10 compact contexts]
    C --> L[Gemini 2.5 Flash]
    L --> A[Markdown answer + node citations]

```

The hypothesis engine allows only technology, tool and vulnerability candidates, obtains related vectors and relationships, and calls Gemini with source context. Evidence linking then looks for confirmation/disconfirmation indicators and updates prediction records; learning uses resolved predictions to store metrics and adjust related node confidence. Historical notes reveal prior quality issues, so this closed loop should be treated as experimental until independently measured.

11. Workflow Documentation

Workflow	Implemented sequence
User research	Open dashboard/search/explore -> retrieve node/graph -> optional research question -> inspect returned node references.
Collection	GitHub cron -> collect.py -> fetch feeds -> de-duplicate checksum -> raw_sources -> job log.
Processing	process.py -> obtain source article where possible -> embedding/classification -> nodes -> mark raw source processed -> flush queue.
Knowledge enrichment	relate -> embed -> cluster -> trend/pattern report; schedules are independent and may consume

Workflow	Implemented sequence
	partially prepared data.
Prediction learning	hypothesis engine -> predictions -> daily evidence linker -> resolved states -> learning metrics/confidence -> periodic model update.
Source maintenance	discover referenced domains -> validate RSS -> sources; dead detector checks/replaces/deactivates; healer repairs graph.
Deployment	Push -> Vercel builds Next.js; scheduled workflows separately execute on GitHub Actions.

```

flowchart LR
    C[Collect: 6h] --> R[raw_sources]
    P[Process: 12h] --> N[nodes]
    R --> P
    N --> L[Relate: daily]
    N --> E[Embed: weekly]
    E --> K[Cluster/Trend: weekly]
    K --> H[Hypotheses: weekly]
    H --> V[Evidence: daily]
    V --> LE[Learning: weekly]
    LE --> MU[Model update: monthly]

```

12. UI / UX Analysis

The visual system is a dark “command center” using Tailwind utility styling, color-coded node types, dense status cards, and Recharts. Main navigation exposes Dashboard, Search, Explore, Timeline, Threats, Patterns, Predictions, Model/Intelligence, Sources, and AI Research. The Header’s global search uses a 200 ms debounce and Ctrl/Cmd+K focus shortcut.

Strengths include clear workflow visibility, source-oriented research citations, and direct navigation between knowledge views. Accessibility gaps include emoji-only/visual status signaling, no evidence of full keyboard focus/ARIA audit, and potential long-live-update complexity. Responsive navigation intentionally hides links below large screens but offers no documented mobile alternative menu.

13. Deployment Analysis

`vercel.json` declares a Next.js framework build using `npm run build` and `.next`. There is no Dockerfile, container orchestration, infrastructure-as-code, migration runner, environment schema, or documented release procedure. GitHub Actions uses `ubuntu-latest`, Python 3.11, dependency installation from `requirements.txt`, cron, and workflow_dispatch. Some workflows have `contents: write` to commit model/cluster artifacts or publish backups.

Production resilience depends on Vercel, Supabase, GitHub Actions and external provider SLAs. Scale guidance: move scheduled compute to managed workers/queues for predictable runtimes, separate public read replicas/views from write privileges, use vector indexes configured for expected corpus size, and centralize observability across web and jobs.

14. Performance Analysis

Area	Bottleneck / improvement
SQLite semantic search	Loads every embedded node and calculates cosine similarity in JS; unsuitable for sizable corpora. Limit local corpus or add a vector extension/index.
Graph traversal	Depth and node count can expand rapidly; current route parses depth without a hard cap. Clamp values, paginate, cache graph neighborhoods.
Embedding API	Remote inference latency/rate limits cause fallback and weak retrieval. Cache embeddings by text hash and monitor provider errors.
Research route	Concurrent retrieval plus Gemini call increases tail latency. Add timeouts, cache frequent queries, stream response, and report retrieval quality.
Batch jobs	Independent cron jobs can overlap and process stale/incomplete input. Use locks, checkpoints and dependency-aware orchestration.
Queries	Several global search/source matches use broad ILIKE patterns. Use dedicated FTS, normalized source domains and indexes.

15. Software Testing

The repository declares pytest but contains no automated test files and no JavaScript test framework/script. Existing `test\_tech.py` is a diagnostic script, not a test suite. Therefore no unit, integration, API contract, UI, migration, security, load, or end-to-end test coverage can be substantiated from the workspace.

Test layer	High-value test cases
Unit	Classifier weight parity with Python; embedding provider/fallback behavior; archived-node filter; cosine edge cases; health-window classification.
API	Schema validation, bad numeric bounds, 404s, opaque 500 errors, research rate limits across instances, prompt/citation output schema.
Database	All declared/used tables/RPCs exist; RLS policies; migration smoke test; vector dimension and relationship invariants.
Pipeline	Idempotent collection/process, duplicate prevention, failure/retry, non-empty embeddings, no self-links, archival policy.
E2E	Browse dynamic IDs, search, source detail, graph cap, research source links, mobile navigation and keyboard flow.
Security/performance	Authz matrix, RLS bypass attempts, prompt injection corpus fixtures, query abuse, concurrent job lock, load test search.

16. Code Quality Review

Strengths: the codebase has a clear separation between presentation, API/data access, and batch jobs; descriptive function names; a useful local fallback; historical audit documentation; and job logging. The route handlers are deliberately small and the classification algorithm is implemented transparently in TypeScript for deployment compatibility.

Technical debt: schema drift (``learning_metrics`` and a batch metadata RPC are referenced but not migrated); widespread ``any`` in TypeScript; parallel Supabase/SQLite behavior drift; mixed data lifecycle policies; hardcoded thresholds/windows; comments that no longer match implementation; absent tests; no formal contracts; and no shared validation/error abstraction. ``db.ts`` is becoming a large multipurpose service that would benefit from typed repositories or bounded domain services.

17. Future Roadmap

Horizon	Prioritized work
Immediate	Restore least-privilege data access; add authentication/roles; make supplied migrations complete; remove public error detail; validate/bound all API inputs; replace hash embedding fallback.
Near term	Add tests and CI gates; introduce migration/version management; route logs to centralized observability; enforce graph/embedding/prediction quality invariants.
Medium term	Durable pipeline orchestration with locks and dependency state; source quality scoring and evidence provenance; user feedback loop for classifications/predictions.
Long term	Tenant-aware intelligence workspaces, analyst review queues, explainable graph confidence, scalable vector retrieval, and governed model evaluation.

18. Conclusion

CyberTree is a credible full-stack cybersecurity intelligence prototype with an unusually broad autonomous pipeline: it collects, enriches, relates, embeds, clusters, forecasts, maintains and exposes knowledge through a polished dashboard. Its core design is understandable and maintainable, especially the database-centered separation between batch processing and web access.

The most important next step is hardening rather than feature growth. The current lack of authentication/authorization, disabled RLS, incomplete migration package, weak semantic fallback, and missing automated tests prevent a production-security assessment above prototype/early pilot maturity. With those foundations corrected, the platform can evolve into a defensible analyst-facing system.

19. Bibliography and References

- Project source snapshot: package.json, README.md, schema.sql, supabase_setup.sql, src/, scripts/, .github/workflows/, STRUCTURE.md, STRUCTURE_V2.md, NOTE.md (analyzed 17 July 2026).
- Next.js documentation: App Router and Route Handlers.
- Supabase documentation: JavaScript client, PostgreSQL, Row Level Security, pgvector integration.
- PostgreSQL documentation: full-text search and GIN indexes; pgvector documentation: vector similarity search.
- MITRE ATT&CK, CAPEC, CWE, NVD and CISA public cybersecurity knowledge resources referenced by ingestion scripts.
- OWASP Top 10 (2021) and OWASP guidance on access control, configuration, injection and logging.
- Hugging Face sentence-transformers `all-MiniLM-L6-v2` model documentation; Google Gemini API documentation.

Appendix A. Complete Project Map

The following inventory includes every accessible, non-generated source/configuration/document file discovered in the repository. Excluded directories were node_modules, venv, .next, data_cache and binary model/database assets except where their purpose is evident from names and calling code.

Path group	Files / purpose
Root configuration	package.json/package-lock.json (Node dependencies); requirements.txt (Python dependencies); next.config.ts; tsconfig.json; eslint.config.mjs; postcss.config.mjs; vercel.json; .gitignore; .env variants (not reproduced to avoid secret disclosure).
Root documentation/schema	README.md; NOTE.md; STRUCTURE.md; STRUCTURE_V2.md; phase_12_predictions_audit.md; schema.sql; supabase_setup.sql.
Assets/models/data	public/*.svg baseline icons; models/classifier.json and classifier.pkl trained artifacts; models/cluster_summary.json; models/training_data.csv and training_report.txt; cyber_tree_local.db local fallback store.
src/app	layout.tsx, globals.css, page.tsx; UI routes: explore, intelligence, patterns, predictions, research, search, similar, sources, threats, timeline.
src/app/api	classify, global-search, graph, intelligence, nodes/[id], nodes, patterns, predictions/[id], predictions, research, similar/[id], sources/[id], sources, stats, system-health.
src/components and lib	Header.tsx; db.ts; embeddings.ts; classifier.ts.
scripts	backup, classify, cluster_nodes, collect, db_client, dead_source_detector, embed_nodes, evidence_linker, export_training_data, hypothesis_engine, ingest_capec/cisa/cwe/mitre/nvd, knowledge_healer, learning_engine, maintain, model_updater, monitor, pattern_report, process, relabel_and_train, relate, research_assistant, source_discoverer, test_tech, train_classifier, trend_detector.
.github/workflows	backup, classify, collect, embed, health_check, hypothesize, learn, maintain, monitor, patterns, process, relate: cron/manual automation.

Appendix B. Verification and Assumptions

- Every non-excluded text/code/configuration file was inventoried and the executable modules, APIs, schema, workflows and documentation were inspected. Binary/database/model artifacts were not reverse engineered; their documented/call-site roles are reported.
- No production credential, database, Vercel, GitHub Actions or external provider connection was made. Historical production outcomes quoted by repository audits are treated as historical evidence, not current measurements.
- The report identifies ``learning_metrics`` and ``update_node_metadata_batch`` as deployment dependencies because source calls them even though supplied migration files do not create them.
- Mermaid diagrams are supplied as portable diagram source in monospace blocks because DOCX does not natively render Mermaid. They can be pasted into a Mermaid renderer for visual export.